Day 19: Transactional Outbox, Dual-Write Safety & Reliable Event Delivery

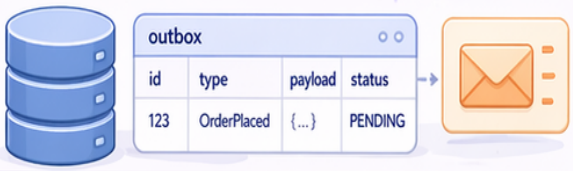
Safely storing state changes and event intent together, then publishing reliably.

1 Why Dual Writes Fail

- DB write succeeds, broker publish fails
- Broker publish succeeds, DB write fails
- State and events can diverge

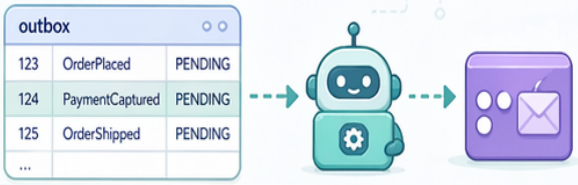
2 Transactional Outbox

- Save domain state + outbox row in one local transaction
- Commit = both records exist
- Abort = neither exists



3 Relay / Poller

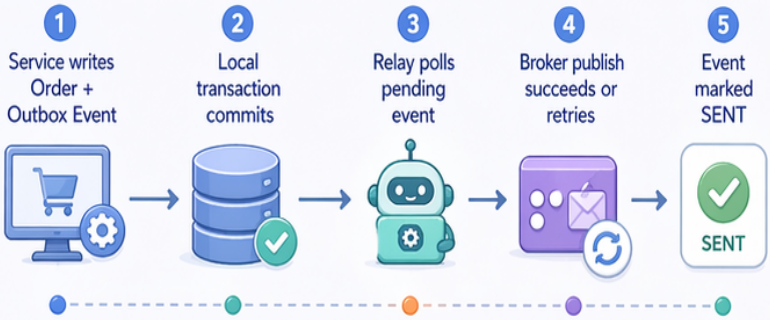
- Poll pending outbox rows
- Publish to broker
- Mark as SENT only after success



4 Delivery Semantics

- At-least-once delivery
- Retries can create duplicates
- Consumers should be idempotent

5 Sample Outbox Flow



6 Trade-Offs

- Fixes service-level dual-write inconsistency
- Adds relay, monitoring, and retention work
- Ordering is easier per key than globally

Transactional outbox keeps state change and event intent atomic, then delivers events reliably with retries and idempotent consumers.

Day 19: Transactional Outbox, Dual-Write Safety, and Reliable Event Delivery

Day 19 focuses on how distributed systems safely publish events when business state changes, without losing updates or emitting ghost events.

It covers:

- dual-write failure mode fundamentals
- transactional outbox pattern
- relay/poller based event publication
- at-least-once delivery and retry behavior
- idempotency implications for consumers
- ordering and throughput trade-offs

In short, Day 19 is about guaranteeing that "state change" and "event intent" are stored atomically, then published reliably.

Core ideas

Why dual writes fail

- Writing to the database and broker in two separate operations can diverge.
- If DB commit succeeds but broker publish fails, downstream systems miss the event.
- If broker publish succeeds but DB commit fails, downstream systems see an event for data that does not exist.

Transactional outbox pattern

- Service writes domain state and an outbox row in one local DB transaction.
- If the transaction commits, both records exist.
- If it aborts, neither exists.

Relay/poller role

- A separate relay process scans pending outbox rows.
- It publishes them to the broker and marks them as sent only after acknowledgement.
- Failed publishes remain pending and are retried.

Delivery semantics

- Outbox relay typically gives at-least-once delivery.
- Duplicate events are possible during retries and crash recovery.
- Consumers should be idempotent and use event IDs for deduplication.

Trade-offs

- Outbox removes dual-write inconsistency for service-owned state.
- It adds operational components (relay, retention, monitoring).
- Ordering is easiest per aggregate/partition key, harder globally.

Day-19 sample: Transactional Outbox Simulation

This repository includes a small transactional outbox sample in both Python and Java.

Functional requirements

- Create an order and outbox event in one local transaction
- Ensure failed transactions do not leak events
- Publish pending outbox events through a relay
- Retry failed publishes without losing messages
- Mark messages as SENT only after publish success

High-level components

- InMemoryStore: domain state and outbox storage
- Transaction: stages order + event and commits atomically
- OutboxRelay: polls pending messages and publishes them
- UnreliableBroker: simulates publish failures for retry demonstration

Data flow

1. Service starts a transaction
2. Service stages domain write and outbox message
3. On commit success, both are persisted together
4. Relay polls pending outbox messages
5. Relay publishes to broker
6. On success, message is marked sent; on failure, attempts increment and message stays pending

Project structure

```
Day-19/
  README.md
  outbox-sample/
    python/
      outbox_pattern.py
      demo.py
    java/
      TransactionalOutbox.java
      Main.java
```

Run the sample

Python

```
cd outbox-sample\python
python demo.py
```

Java

```
cd outbox-sample\java
javac Main.java TransactionalOutbox.java
java Main
```

Interview takeaways

- Transactional outbox is the standard fix for service-level dual-write inconsistency
- Relay failures should not delete events; retries preserve eventual delivery
- At-least-once delivery means idempotent consumers are mandatory
- Monitoring pending outbox backlog is a key operational SLI
- Outbox pairs well with Saga, CQRS, and event-driven architectures

Next improvements

- Add partition-aware relay workers for higher throughput
- Add deduplication keys and consumer-side idempotency store
- Replace polling with CDC (for example, Debezium-based streaming)
- Add retention/archival policy for sent outbox rows
- Add metrics for lag, retry count, and publish failure rate

Code Appendix

Python and Java source used in Day 19 sample.

outbox-sample\python\outbox_pattern.py

```
from __future__ import annotations

from dataclasses import dataclass
from enum import Enum


class OutboxStatus(Enum):
    PENDING = "PENDING"
    SENT = "SENT"


@dataclass(frozen=True)
class Order:
    order_id: str
    amount_cents: int


@dataclass
class OutboxMessage:
    message_id: int
    aggregate_id: str
    topic: str
    payload: str
    status: OutboxStatus = OutboxStatus.PENDING
    attempts: int = 0


class InMemoryStore:
    def __init__(self) -> None:
        self._orders: dict[str, Order] = {}
        self._outbox: list[OutboxMessage] = []
        self._next_message_id = 1

    def begin_transaction(self) -> Transaction:
        return Transaction(self)

    def _allocate_message_id(self) -> int:
        message_id = self._next_message_id
        self._next_message_id += 1
        return message_id

    def pending_messages(self, limit: int) -> list[OutboxMessage]:
        if limit <= 0:
            return []
        pending = [m for m in self._outbox if m.status == OutboxStatus.PENDING]
        return pending[:limit]

    def mark_sent(self, message_id: int) -> None:
        message = self._find_message(message_id)
        message.status = OutboxStatus.SENT

    def mark_failure(self, message_id: int) -> None:
        message = self._find_message(message_id)
        message.attempts += 1

    def _find_message(self, message_id: int) -> OutboxMessage:
        for message in self._outbox:
            if message.message_id == message_id:
                return message
        raise KeyError(f"unknown message_id: {message_id}")

    def orders_snapshot(self) -> dict[str, Order]:
        return dict(self._orders)
```

```
    def outbox_snapshot(self) -> list[OutboxMessage]:
        return [
            OutboxMessage(
                message_id=m.message_id,
                aggregate_id=m.aggregate_id,
                topic=m.topic,
                payload=m.payload,
                status=m.status,
                attempts=m.attempts,
            )
            for m in self._outbox
        ]


class Transaction:
    def __init__(self, store: InMemoryStore) -> None:
        self._store = store
        self._staged_order: Order | None = None
        self._staged_event: OutboxMessage | None = None

    def create_order(self, order_id: str, amount_cents: int) -> None:
        if order_id in self._store._orders:
            raise ValueError(f"order already exists: {order_id}")
        if amount_cents <= 0:
            raise ValueError("amount_cents must be positive")

        self._staged_order = Order(order_id=order_id, amount_cents=amount_cents)
        self._staged_event = OutboxMessage(
            message_id=self._store._allocate_message_id(),
            aggregate_id=order_id,
            topic="orders.created",
            payload=f"{{'order_id':'{order_id}','amount_cents':{amount_cents}}}",
        )

    def commit(self, simulate_db_failure: bool = False) -> None:
        if self._staged_order is None or self._staged_event is None:
            raise ValueError("transaction has no staged work")
        if simulate_db_failure:
            raise RuntimeError("simulated database failure before commit")

        self._store._orders[self._staged_order.order_id] = self._staged_order
        self._store._outbox.append(self._staged_event)


class UnreliableBroker:
    def __init__(self, fail_on_calls: set[int] | None = None) -> None:
        self._fail_on_calls = fail_on_calls or set()
        self._call_count = 0
        self.published: list[str] = []

    def publish(self, topic: str, payload: str) -> bool:
        self._call_count += 1
        if self._call_count in self._fail_on_calls:
            return False
        self.published.append(f"{topic}:{payload}")
        return True


class OutboxRelay:
    def __init__(self, store: InMemoryStore, broker: UnreliableBroker) -> None:
        self._store = store
        self._broker = broker

    def poll_and_publish(self, limit: int = 10) -> list[str]:
        history: list[str] = []
        for message in self._store.pending_messages(limit):
            ok = self._broker.publish(message.topic, message.payload)
            if ok:
                self._store.mark_sent(message.message_id)
                history.append(f"published message {message.message_id}")
            else:
```

```
                self._store.mark_failure(message.message_id)
                history.append(
                    f"publish failed for message {message.message_id}; will retry"
                )
        return history
```

outbox-sample\python\demo.py

```
from outbox_pattern import InMemoryStore, OutboxRelay, UnreliableBroker


def print_state(store: InMemoryStore) -> None:
    print("orders:")
    for order_id, order in store.orders_snapshot().items():
        print(f"  {order_id} amount={order.amount_cents}")

    print("outbox:")
    for message in store.outbox_snapshot():
        print(
            "  "
            f"id={message.message_id} aggregate={message.aggregate_id} "
            f"status={message.status.value} attempts={message.attempts}"
        )


def main() -> None:
    store = InMemoryStore()

    tx1 = store.begin_transaction()
    tx1.create_order(order_id="order-100", amount_cents=2599)
    tx1.commit()

    tx2 = store.begin_transaction()
    tx2.create_order(order_id="order-101", amount_cents=1299)
    try:
        tx2.commit(simulate_db_failure=True)
    except RuntimeError as error:
        print("simulated failure:", error)

    print("\nstate after transactions")
    print_state(store)

    broker = UnreliableBroker(fail_on_calls={1})
    relay = OutboxRelay(store=store, broker=broker)

    print("\nrelay pass 1")
    for line in relay.poll_and_publish(limit=10):
        print(" ", line)
    print_state(store)

    print("\nrelay pass 2")
    for line in relay.poll_and_publish(limit=10):
        print(" ", line)
    print_state(store)


if __name__ == "__main__":
    main()
```

outbox-sample\java\TransactionalOutbox.java

```
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;

public class TransactionalOutbox {
    public enum OutboxStatus {
        PENDING,
```

```
    SENT
}

public static final class Order {
    private final String orderId;
    private final int amountCents;

    public Order(String orderId, int amountCents) {
        this.orderId = orderId;
        this.amountCents = amountCents;
    }

    public String getOrderId() {
        return orderId;
    }

    public int getAmountCents() {
        return amountCents;
    }
}

public static final class OutboxMessage {
    private final int messageId;
    private final String aggregateId;
    private final String topic;
    private final String payload;
    private OutboxStatus status;
    private int attempts;

    public OutboxMessage(int messageId, String aggregateId, String topic, String payload) {
        this.messageId = messageId;
        this.aggregateId = aggregateId;
        this.topic = topic;
        this.payload = payload;
        this.status = OutboxStatus.PENDING;
        this.attempts = 0;
    }

    public int getMessageId() {
        return messageId;
    }

    public String getAggregateId() {
        return aggregateId;
    }

    public String getTopic() {
        return topic;
    }

    public String getPayload() {
        return payload;
    }

    public OutboxStatus getStatus() {
        return status;
    }

    public int getAttempts() {
        return attempts;
    }

    private void markSent() {
        status = OutboxStatus.SENT;
    }

    private void markFailure() {
        attempts += 1;
    }

    public OutboxMessage copy() {
```

```
        OutboxMessage cloned = new OutboxMessage(messageId, aggregateId, topic, payload);
        cloned.status = status;
        cloned.attempts = attempts;
        return cloned;
    }
}

public static final class InMemoryStore {
    private final Map<String, Order> orders = new LinkedHashMap<>();
    private final List<OutboxMessage> outbox = new ArrayList<>();
    private int nextMessageId = 1;

    public Transaction beginTransaction() {
        return new Transaction(this);
    }

    public List<OutboxMessage> pendingMessages(int limit) {
        if (limit <= 0) {
            return List.of();
        }
        List<OutboxMessage> pending = new ArrayList<>();
        for (OutboxMessage message : outbox) {
            if (message.getStatus() == OutboxStatus.PENDING) {
                pending.add(message);
                if (pending.size() >= limit) {
                    break;
                }
            }
        }
        return pending;
    }

    public void markSent(int messageId) {
        findMessage(messageId).markSent();
    }

    public void markFailure(int messageId) {
        findMessage(messageId).markFailure();
    }

    public Map<String, Order> ordersSnapshot() {
        return new LinkedHashMap<>(orders);
    }

    public List<OutboxMessage> outboxSnapshot() {
        List<OutboxMessage> snapshot = new ArrayList<>();
        for (OutboxMessage message : outbox) {
            snapshot.add(message.copy());
        }
        return snapshot;
    }

    private int allocateMessageId() {
        int id = nextMessageId;
        nextMessageId += 1;
        return id;
    }

    private OutboxMessage findMessage(int messageId) {
        for (OutboxMessage message : outbox) {
            if (message.getMessageId() == messageId) {
                return message;
            }
        }
        throw new IllegalArgumentException("unknown message_id: " + messageId);
    }
}

public static final class Transaction {
    private final InMemoryStore store;
    private Order stagedOrder;
```

```
    private OutboxMessage stagedEvent;

    private Transaction(InMemoryStore store) {
        this.store = store;
    }

    public void createOrder(String orderId, int amountCents) {
        if (store.orders.containsKey(orderId)) {
            throw new IllegalArgumentException("order already exists: " + orderId);
        }
        if (amountCents <= 0) {
            throw new IllegalArgumentException("amountCents must be positive");
        }

        stagedOrder = new Order(orderId, amountCents);
        stagedEvent = new OutboxMessage(
            store.allocateMessageId(),
            orderId,
            "orders.created",
            "{'order_id':'" + orderId + "','amount_cents':" + amountCents + "}"
        );
    }

    public void commit(boolean simulateDbFailure) {
        if (stagedOrder == null || stagedEvent == null) {
            throw new IllegalStateException("transaction has no staged work");
        }
        if (simulateDbFailure) {
            throw new IllegalStateException("simulated database failure before commit");
        }

        store.orders.put(stagedOrder.getOrderId(), stagedOrder);
        store.outbox.add(stagedEvent);
    }
}

public interface Broker {
    boolean publish(String topic, String payload);
}

public static final class UnreliableBroker implements Broker {
    private final Set<Integer> failOnCalls;
    private final List<String> published = new ArrayList<>();
    private int callCount = 0;

    public UnreliableBroker(Set<Integer> failOnCalls) {
        this.failOnCalls = new LinkedHashSet<>(failOnCalls);
    }

    @Override
    public boolean publish(String topic, String payload) {
        callCount += 1;
        if (failOnCalls.contains(callCount)) {
            return false;
        }
        published.add(topic + ":" + payload);
        return true;
    }

    public List<String> getPublished() {
        return List.copyOf(published);
    }
}

public static final class OutboxRelay {
    private final InMemoryStore store;
    private final Broker broker;

    public OutboxRelay(InMemoryStore store, Broker broker) {
        this.store = store;
        this.broker = broker;
```

```
        }

        public List<String> pollAndPublish(int limit) {
            List<String> history = new ArrayList<>();
            for (OutboxMessage message : store.pendingMessages(limit)) {
                boolean ok = broker.publish(message.getTopic(), message.getPayload());
                if (ok) {
                    store.markSent(message.getMessageId());
                    history.add("published message " + message.getMessageId());
                } else {
                    store.markFailure(message.getMessageId());
                    history.add(
                        "publish failed for message "
                            + message.getMessageId()
                            + "; will retry"
                    );
                }
            }
            return history;
        }
    }
}
```

outbox-sample\java\Main.java

```
import java.util.Map;
import java.util.Set;

public class Main {
    public static void main(String[] args) {
        TransactionalOutbox.InMemoryStore store = new TransactionalOutbox.InMemoryStore();

        TransactionalOutbox.Transaction tx1 = store.beginTransaction();
        tx1.createOrder("order-100", 2599);
        tx1.commit(false);

        TransactionalOutbox.Transaction tx2 = store.beginTransaction();
        tx2.createOrder("order-101", 1299);
        try {
            tx2.commit(true);
        } catch (IllegalStateException error) {
            System.out.println("simulated failure: " + error.getMessage());
        }

        System.out.println("\nstate after transactions");
        printState(store);

        TransactionalOutbox.UnreliableBroker broker = new TransactionalOutbox.UnreliableBroker(Set.of(1));
        TransactionalOutbox.OutboxRelay relay = new TransactionalOutbox.OutboxRelay(store, broker);

        System.out.println("\nrelay pass 1");
        for (String line : relay.pollAndPublish(10)) {
            System.out.println("  " + line);
        }
        printState(store);

        System.out.println("\nrelay pass 2");
        for (String line : relay.pollAndPublish(10)) {
            System.out.println("  " + line);
        }
        printState(store);
    }

    private static void printState(TransactionalOutbox.InMemoryStore store) {
        System.out.println("orders:");
        for (Map.Entry<String, TransactionalOutbox.Order> entry : store.ordersSnapshot().entrySet()) {
            System.out.println(
                "  "
                    + entry.getKey()
                    + " amount="
                    + entry.getValue().getAmountCents()
```

```
            );
        }

        System.out.println("outbox:");
        for (TransactionalOutbox.OutboxMessage message : store.outboxSnapshot()) {
            System.out.println(
                "  id="
                    + message.getMessageId()
                    + " aggregate="
                    + message.getAggregateId()
                    + " status="
                    + message.getStatus()
                    + " attempts="
                    + message.getAttempts()
            );
        }
    }
}
```